



Meta final - Base de Dados

Metro Mondego System

Diogo Saldanha | uc2022232761@student.uc.pt | PL7

João Barata | uc2022245160@student.uc.pt | PL7

Vitor Cruz | uc2017278238@student.uc.pt | PL7

Breve descrição do projeto

O projeto consiste no desenho, implementação e teste de um sistema de gestão de base de dados relacional (PostgreSQL) para o **Metro Mondego**, exposto através de uma REST API desenvolvida em Python (Flask).

O sistema modela a operação logística e comercial de três linhas fixas na região de Coimbra:

1. **Linha 1:** Portagem - Hospital (Urbano)
2. **Linha 2:** Portagem - Estação B (Urbano)
3. **Linha 3:** Portagem - Miranda do Corvo - Lousã (Serpins) (Regional)

A aplicação gere múltiplos perfis de utilizador (Super Administrador, Administrador e Cliente) com permissões verticais estritas. Os clientes podem carregar a sua carteira digital, adquirir títulos de viagem (bilhetes simples, diários ou passes mensais), aplicar regras de desconto promocionais automáticas e validar as suas viagens nas estações da rede. Para além disso, o backend disponibiliza endpoints analíticos complexos (demand, top spenders e monthly reports) processados inteiramente através de queries SQL nativas no servidor de BD, respeitando a restrição de não utilização de ORMs ou funções de janela proibidas.

Manual de Instalação

Pré-requisitos

- **PostgreSQL Engine** instalado localmente.
- **Python 3.x** com o gestor de pacotes pip instalado.
- **Postman** para a execução da suite de testes.

Configuração e Inicialização da Base de Dados

1. Abra o utilitário de administração do PostgreSQL (ex: pgAdmin).
2. Crie uma base de dados vazia chamada projeto.
3. Execute o script SQL contido no ficheiro metafinal.sql para efetuar a limpeza de esquemas antigos, criar as novas tabelas com restrições de integridade e popular as tabelas com os dados semente (*seed data*) exigidos (Linhas, Configuração Operacional, Tipos de Produto e Preços Base do Tarifário).

Configuração do Servidor Web (Python)

1. Certifique-se de que possui as bibliotecas necessárias instaladas:

```
pip install flask psycopg2 PyJWT
```

2. No ficheiro metro.py, valide as credenciais da função de ligação à base de dados (db_connection) com a sua password local do PostgreSQL:

```
def db_connection():
    db = psycopg2.connect(
        user='postgres',
        password='YOUR_PASSWORD_HERE', # Atualizar conforme necessário
        host='127.0.0.1',
        port='5432',
        database='projeto'
    )
    return db
```

3. Inicie o servidor executando o script:

```
python metro.py
```

O servidor estará à escuta no endereço `http://127.0.0.1:8080`.

Manual de utilizador (postman)

A validação das funcionalidades da API é realizada importando a coleção JSON fornecida no Postman. O fluxo de testes deve seguir a ordem estrita dos pedidos devido às dependências de autorização (Bearer Tokens):

A nossa suite de testes no Postman valida o funcionamento da API através de variáveis de ambiente dinâmicas (`baseUrl`, `adminToken`, `customerToken`, etc.), permitindo testar o fluxo completo do sistema sequencialmente.

Abaixo detalha-se o propósito de cada um dos 14 pedidos configurados na coleção:

Bloco 1: Autenticação e Gestão de Utilizadores

- **1. Buscar token (PUT /dbproj/user):** Autentica um utilizador no sistema (ex: Super Administrador) usando as suas credenciais e devolve um token JWT para autorizar os pedidos seguintes.
- **2. Add admin (PUT /dbproj/register/admin):** Permite ao Super Administrador registar um novo perfil de administrador na plataforma.
- **3. Add client (POST /dbproj/register/customer):** Permite o registo de um cliente comum, validando dados únicos como NIF e telefone, e inicializando a sua carteira digital a zero.

Bloco 2: Configuração e Operação (Administradores)

- **4. Update OperaçãoLinha (PUT /dbproj/line_operation/{id}):** Altera as definições de funcionamento de uma linha, nomeadamente a hora de início/fim, a frequência e a lotação máxima dos veículos.
- **5. Atualizar tarifário (PUT /dbproj/fares/{id}):** Regista um novo preço para um tipo de bilhete a partir de uma data específica, assegurando o fecho automático da vigência do preço anterior no histórico.
- **6. Notice Broadcast (POST /dbproj/notices/broadcast):** Emite um aviso global de serviço (ex: alertas de greve ou atrasos) visível para todos os clientes da rede.
- **7. Create Promotion (POST /dbproj/promotions):** Define uma regra de desconto temporária (em percentagem) aplicável a um determinado produto e linha.

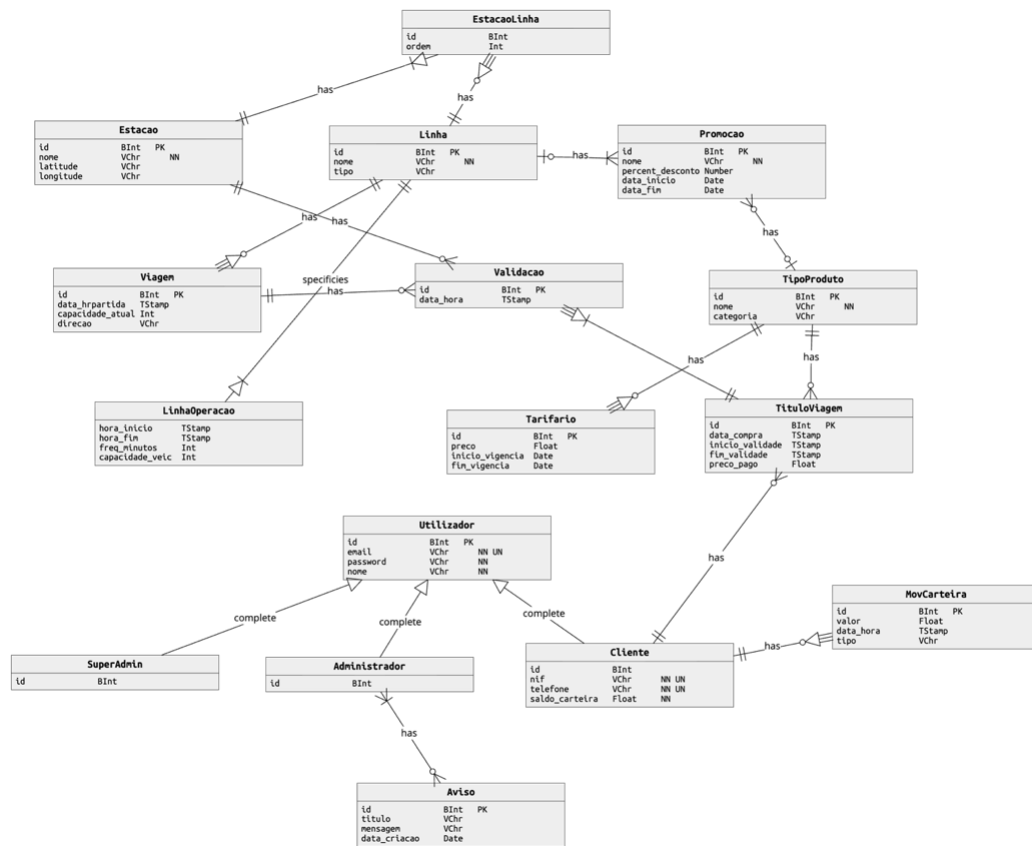
Bloco 3: Ações do Cliente

- **8. List Lines (GET /dbproj/lines_next):** Lista em tempo real as próximas viagens agendadas para todas as linhas, calculando dinamicamente a capacidade livre em cada veículo.
- **9. Add Wallet Funds (POST /dbproj/wallet/topup):** Simula o carregamento monetário da carteira digital do cliente autenticado, registrando o movimento na base de dados.
- **10. Purchase Ticket (POST /dbproj/purchase):** Efetua a compra de um título de viagem deduzindo o custo no saldo do cliente. O sistema aplica descontos promocionais ativos automaticamente e o Postman guarda o `purchase_id` gerado.
- **11. Use Ticket (POST /dbproj/ticket/use/{id}):** Simula a validação do bilhete recém-comprado numa estação física. O endpoint valida a janela temporal, a linha da estação, impede reutilizações de bilhetes simples e verifica se o veículo não excedeu a lotação.

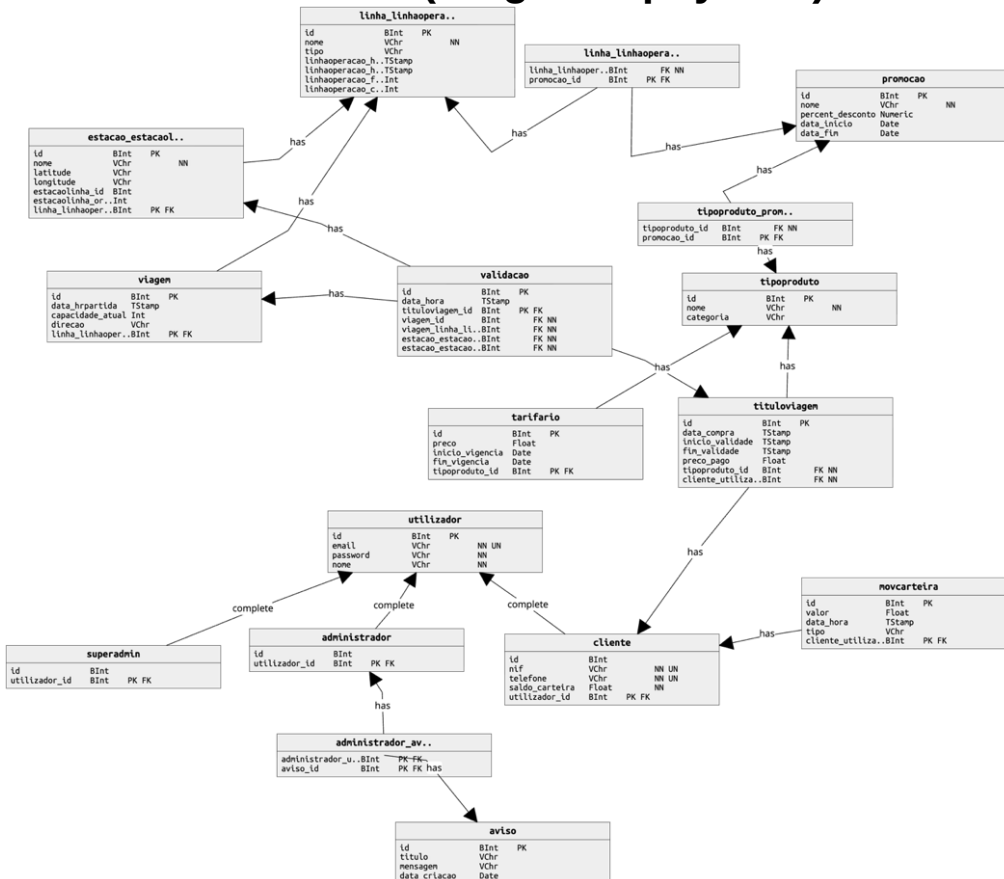
Bloco 4: Relatórios Analíticos

- **12. Demand Report (GET /dbproj/report/demand):** Retorna as janelas horárias com o maior (pico) e menor número de validações registradas, agrupadas por linha.
 - **13. Top Spenders Report (GET /dbproj/report/top_spenders):** Identifica os clientes individuais que gastaram mais fundos em títulos de viagem nos últimos 30 dias, por cada linha operada.
 - **14. Monthly Report (GET /dbproj/report/monthly):** Fornece uma estatística mensal agregada por linha, indicando o número de clientes ativos e quantos destes foram recorrentes (2 ou mais validações).
-

Modelo de dados (Diagrama conceptual)



Modelo de dados (Diagrama physical)



Destaques de Modelação no Esquema Físico:

- **Estratégia de Herança (Tabela Pai-Filho):** Implementou-se uma herança de tabelas através de restrições de chaves estrangeiras (REFERENCES utilizador(id) ON DELETE CASCADE) nas tabelas super_admin, administrador e cliente. Isto evita a duplicação de dados e isola atributos específicos de forma limpa (ex: o saldo_carteira, nif e telefone pertencem estritamente à tabela cliente).
- **Associações N:M Explícitas:** A tabela de ligação estacao_linha mapeia a relação entre estações e linhas salvaguardando a propriedade crítica da ordem das paragens ao longo do trajeto.

Definição de Transações e Concorrência

Para salvaguardar a integridade dos dados e evitar anomalias sob execução concorrente massiva, foram aplicadas as seguintes estratégias no servidor de base de dados PostgreSQL e no código Python:

Isolamento Prévio e Bloqueio Pessimista (FOR UPDATE)

No endpoint de validação de títulos (/dbproj/ticket/use/<int:ticket_id>), existe o risco iminente de concorrência se um cliente tentar validar o mesmo bilhete simples em múltiplos validadores em simultâneo (ataque de duplo gasto). Para mitigar este problema:

1. É executado um bloqueio explícito da linha correspondente ao título de viagem recorrendo à cláusula **FOR UPDATE OF tv**. Isto impede que outras transações paralelas leiam ou alterem o estado de validade do bilhete até que a validação atual termine.
2. Adicionalmente, na seleção da viagem concorrente, utiliza-se a cláusula **FOR UPDATE** na tabela viagem para congelar o registo enquanto se faz a contagem agregada de passageiros (COUNT(val.id)), prevenindo que a capacidade máxima do veículo configurada pelo administrador seja ultrapassada em frações de segundo por requisições concorrentes.

Gestão Atômica de Transações (Rollback e Commit)

Em operações que envolvem escritas multi-tabela (como a criação de utilizadores em add_admin e add_customer, que inserem primeiro na tabela pai utilizador e depois na filha), o fluxo de controlo utiliza blocos try/except. Qualquer violação de restrição de integridade (como chaves duplicadas em email, nif ou telefone) dispara um **conn.rollback()**, garantindo que a base de dados nunca fique num estado inconsistente. Os dados só são persistidos permanentemente após um **conn.commit()** bem-sucedido.

Processamento Analítico Server-Side Ponto a Ponto

Como estipulado nas restrições técnicas do projeto, os endpoints analíticos não utilizam processamento em Python nem operadores de janela proibidos (como RANK ou ROW_NUMBER).

- **Peak and Low Demand (/dbproj/report/demand):** Utiliza uma única query baseada em subqueries agregadas comparativas com MAX e MIN para isolar os intervalos horários de maior e menor afluência por linha.
 - **Top Spenders (/dbproj/report/top_spenders):** Agrega a soma de gastos (SUM(preco_pago)) filtrada pelos últimos 30 dias e utiliza uma subquery correlacionada para extrair o cliente com o gasto máximo absoluto por linha.
-

7. Relatório de Execução e Esforço

A divisão inicial de tarefas e o esforço individual foram balanceados de forma a assegurar que todos os membros compreendem a totalidade da arquitetura de dados e de rede implementada.

Tarefa / Funcionalidade	Responsável Principal
Modelação ER e Relacional (ONDA)	Todos
Escrita do Esquema DDL SQL (metafinal.sql)	João Barata
Implementação das rotas de Autenticação e Registos (1, 2, 3)	João Barata
Desenvolvimento das rotas de Operação e Compra (4, 5, 6, 7, 10)	Todos
Lógica de Validação e Concorrência (8, 9, 11)	Vitor Cruz / Diogo Saldanha
Construção das Queries Analíticas Complexas (12, 13, 14)	Diogo Saldanha
Testes Automatizados no Postman e Documentação	Todos

Nota Final: Ambas as partes colaboraram ativamente em sessões de *pair programming* para debug nos conflitos transacionais e garantir a conformidade dos dados retornados com as assinaturas JSON estritas exigidas no enunciado do projeto.